

Тема 3. Организация на кода. Модули и функции – дефиниции, атрибути, предаване на параметри, видимост

1. Организиране на кода

Кодът в Elixir се организира чрез **модули и функции**, което подобрява четимостта, повторната употреба и поддръжката. Всеки модул групира свързана функционалност и дефинира ясни граници на отговорност.

1.1. Функции *(разгледано в лекция 2)*

1.2. Макроси

В рамките на един и същи модул в Elixir могат да съществуват едновременно, както функции така и макроси. Макросите в Elixir са механизъм за **метапрограмиране**, който позволява кодът да генерира или трансформира друг код по време на компилация. Макросът получава кода като структура (AST – Abstract Syntax Tree), променя го и връща нов код, който след това се компилира.

Основни правила за наименоуване на макроси

- Името започва с малка буква или `_`.
- Може да съдържа малки букви, цифри и `_`.
- Може да завършва със `?` или `!` (по конвенция за въпроси или функции с възможна грешка).
- Дефинира се с ключовата дума `defmacro`.
- След името се указва arity (брой аргументи).

Синтаксис:

```
defmodule MyMacro do
  defmacro say_hello(name) do
    quote do
      IO.puts("Hello #{unquote(name)}")
    end
  end
end

//Използване
require MyMacro
MyMacro.say_hello("Ivan")
```

- `defmacro` – дефинира макрос.
- `quote` – превръща кода в AST (код като данни).
- `unquote` – вмъква стойността на аргумента в генерирания код.
- `require` MyMacro указва на компилатора, че модулът MyMacro съдържа макроси, които ще бъдат използвани в текущия модул или файл. `require` MyMacro зарежда модула и позволява извикване на неговите макроси по време на компилация. Без `require` извикването на макроса (`MyMacro.hello()`) ще доведе до грешка при компилация.

Реално при компилация макросът генерира:

```
IO.puts("Hello Ivan")
```

Синтаксис

```
defmacro име(аргументи) do
  quote do
    код
  end
end
```

Основни характеристики на макросите

- Изпълняват се по време на компилация, не по време на изпълнение.
- Работят върху AST (Abstract Syntax Tree).
- Използват се чрез `quote` и `unquote`.
- Позволяват създаване на DSL (domain-specific languages).

Пример за реална употреба

Много от конструкциите в Elixir са макроси, например:

```
if x > 0 do
  IO.puts("positive")
end
```

if всъщност е макрос, а не обикновена функция.

Кога се използват макросите?

Когато:

- трябва да се генерира код автоматично
- се създава DSL
- се намалява повторението на код
- се правят compile-time трансформации

Кога се изпълняват макросите и кога функциите?

- Функциите се изпълняват по време на изпълнение (runtime).
- Макросите по време на компилация (compile time).

Пример 1: (функции и макроси в един модул)

```
defmodule Example do
  # функция
  def hello(name) do
    "Здравей, #{name}"
  end

  # макрос
  defmacro say_hello(expr) do
    quote do
      IO.puts("Здравей от макроса!")
    unquote(expr)
    end
  end
end
```

Използване

```
Example.hello("Ана")
require Example
Example.say_hello(IO.puts("Край"))
```

Правила за използване/ дефиниране на макроси и функции в Elixir

- Функции и макроси могат да са в един и същи модул.
- Макросите се дефинират с **defmacro**.
- За използване на макрос е необходим **require** (require зарежда съответния модул и позволява извикване на неговите макроси по време на компилация).
- Макросите *генерират код, а не стойности*.
- Функциите *не могат да се използват като макроси и обратно*.

Каква е разликата между макрос и функция?

Функциите работят със стойности по време на изпълнение, а макросите работят с код по време на компилация.

Таблица 1. Сравнение между функции и макроси

Критерий	Функции	Макроси
Кога се изпълняват?	Runtime (по време на изпълнение)	Compile-time (по време на компилация)
С какво работят?	Със стойности	С код (AST)
Какво връщат?	Стойност	Код
Синтаксис	def	defmacro
Необходим ли е require?	Не	Да

Могат ли да се подават като аргумент?	Да	Не
Каква е основната им цел?	Бизнес логика	Генериране / трансформация на код
Безопасни ли са?	По-безопасни	По-рискови (ако се злоупотребят)

Пример 2: (Функция) *Работи със стойности и връща резултат.*

```
def add(a, b) do
  a + b
end
```

Пример 3: (Макрос) *Генерира нов код по време на компилация.*

```
defmacro say(expr) do
  quote do
    IO.puts("Изпълнявам код:")
    unquote(expr)
  end
end
```

В какви случаи се използва макрос и в какви функция?

Използване на функция, когато:

- се работи с данни,
- се пише логика,
- желаем предсказуемо поведение.

Използване на макрос, когато:

- се изисква нов синтаксис,
- има повтарящ се код,
- има метапрограмиране.

Следователно: В Elixir един модул може да съдържа, както функции, така и макроси, като функциите се изпълняват по време на изпълнение (работят със стойности), а макросите – по време на компилация (работят с код). Макросите се използват за генериране и трансформиране на код, а функциите – за реализиране на логика.

2. Модули

Модулите са основната единица за структуриране на кода в Elixir. Те съдържат наименувани функции и атрибути.

- **Наименование** (*разгледано в лекция 2*)
- **Влагане и наследяване на модули**

1. Влагане на модули (Module Nesting)

Какво представлява влагането на модул в Elixir?

Влагането на модули означава дефиниране на модул вътре в друг модул, като **вложеният** модул **логически принадлежи към външния**.

Синтаксис

```
defmodule Outer do
  defmodule Inner do
    def hello do
      "Здравей"
    end
  end
end
```

Извикване

```
Outer.Inner.hello()
```

Правила за влагане

- Вложеният модул е напълно самостоятелен.
- Няма автоматичен достъп до функции на външния модул.
- Името му става Outer.Inner.
- Използва се за логическо групиране на функционалност.

Пример 4:

```
defmodule Math do
  defmodule Statistics do
    def average(list) do
      Enum.sum(list) / length(list)
    end
  end
end
```

Извикване

```
Math.Statistics.average([1, 2, 3, 4])
```

Функция от вложен модул се извиква чрез пълното име на модула с точкова нотация.

В конкретния случай:

- Math е външният модул.
- Statistics е вложен модул в Math.
- average/1 е наименувана функция във вложения модул.

```
result = Math.Statistics.average([1, 2, 3, 4])
IO.puts(result)
```

Резултатът е:

Резултатът ще бъде 2.5.

2. Наследяване при модули в Elixir (важна особеност!)

Правила за наследяване в Elixir

Elixir **НЕ** поддържа класическо наследяване, както в обектно-ориентираните езици. Вместо това използва:

- **Композиция** – съчетаване на данни и функции (data и functions) за постигане на полиморфизъм.
- **use с макроси** – включване на функционалност от друг модул.
- **Протоколи (protocol)** – дефинират поведение (“интерфейс”) за различни типове данни.
- **Поведение (behaviour)** – дефинира задължителни функции, които модулет трябва да имплементира, подобно на интерфейс в ООП.

2.1. Композиция

Какво е композиция (Composition)?

Композицията е подход за изграждане на функционалност чрез комбиниране на по-малки, независими модули и функции, вместо чрез наследяване. Вместо „е наследник на“ (is-a), композицията прилага „използва / съдържа“ (has-a).

В раките на програмният език Elixir композицията означава:

- използване на функции от други модули,
- подаване на модули като аргументи,
- комбиниране на поведение чрез pipeline (|>),
- структуриране чрез данни и функции, а не чрез йерархии.

Пример 5. //Композиция чрез използване на модули

```
defmodule Math do
  def sum(list), do: Enum.sum(list)
end

defmodule Statistics do
  def average(list) do
    Math.sum(list) / length(list)
  end
end
```

Statistics използва **Math**, но не го наследява.

Извикване

```
Statistics.average([1, 2, 3, 4, 5])  
# => 3.0
```

`Math.sum(list)` се извиква вътре в `Statistics.average/1`, защото `Math` е модулът, който дефинира функцията `sum/1`. Във външния код извикваме само `Statistics.average/1` със списъка.

Извикване в iex с команди:

```
iex> Statistics.average([10, 20, 30])  
20.0
```

Пример 6. //Композиция чрез pipeline оператор

```
list  
> Enum.filter(&(&1 > 0))  
> Enum.sum()
```

Сложна логика се изгражда от малки функции, свързани последователно.

Извикване

```
list = [-2, 5, 0, 3, -1]  
  
# Използване на pipeline  
result = list  
> Enum.filter(&(&1 > 0))      # първо филтрираме само положителните числа  
> Enum.sum()                 # после сумираме филтрираните числа  
  
IO.puts(result) # => 8
```

Функцията `list > Enum.filter(&(&1 > 0))` взема списъка и оставя само елементите `> 0` т.е. `[5, 3]`. Функцията `> Enum.sum()` взема резултата от предишната стъпка `[5, 3]` и го сумира така се получава `8`. Т.е. `pipeline >` взема резултата от предишната стъпка и го подава като първи аргумент на следващата функция.

Пример 7. //Композиция чрез подаване на модул (полиморфизъм)

Да се създаде функция `process(worker_module, data)`, която извиква `worker_module.work(data)`. Да се създадат 2 различни модула (`WorkerA` и `WorkerB`) с функция `work/1`, които обработват данните по различен начин. Да се използва `process/2`, за да се извика съответния модул с подадени данни. Целта е чрез функцията `process/2` да се демонстрира полиморфизъм чрез композиция на модули. Вместо да се използват класове и наследяване, подаваме като аргумент модул, който има функция `work/1`. Това позволява динамично променяне на поведението.

Примерна реализация

```
# Модул WorkerA  
defmodule WorkerA do  
  def work(data) do  
    "WorkerA processed #{data}"  
  end  
end  
  
# Модул WorkerB  
defmodule WorkerB do  
  def work(data) do  
    "WorkerB handled #{data}"  
  end  
end  
  
# Функцията process/2  
defmodule Processor do  
  def process(worker_module, data) do  
    worker_module.work(data)  
  end  
end
```

Подаваме WorkerA

```
Processor.process(WorkerA, "task1")  
# -> WorkerA.work("task1")  
# => "WorkerA processed task1"
```

Подаваме WorkerB

```
Processor.process(WorkerB, "task2")  
# -> WorkerB.work("task2")  
# => "WorkerB handled task2"
```

Таблица 2. Постъпково изпълнение

Стъпка	Какво се случва?
1	process/2 получава модул (WorkerA/ WorkerB) и данни (task1/ task2)
2	Вътре в process, се извиква worker_module.work(data)
3	В зависимост от подадения модул се изпълнява конкретната реализация на work/1
4	Връща се резултатът на извършената обработка

Основни правила за композиция в Elixir

- Функционалността се изгражда от малки функции.
- Модулите са независими и слабо свързани.
- Няма йерархии от класове.
- Данните и поведението са разделени.
- Композицията се предпочита пред наследяването.

Таблица 3. Сравнение между наследяване и композиция в ООП и Elixir

Наследяване (ООП)	Композиция (Elixir)
Йерархия	Комбиниране
Силна зависимост	Слаба зависимост
Повторна употреба чрез клас	Повторна употреба чрез функции

Следователно: Композицията в Elixir е начин за изграждане на програми чрез комбиниране на независими модули и функции, вместо чрез класическо наследяване. Тя води до по-гъвкав, модулен и лесен за поддръжка код.

2.2. „Наследяване“ чрез use (макроси)

- use позволява включване на код от друг модул чрез макроси, което наподобява наследяване.

Пример 8: Създайте макрос в модул LoggerBase, който чрез `__using__/1` добавя функция `log/1` към всеки модул, който използва `use LoggerBase`. След това дефинирайте модул MyApp, който използва този макрос и позволява извикване на функцията `log/1` за извеждане на съобщение с префикс "LOG:".

```
defmodule LoggerBase do  
  defmacro __using__(_opts) do  
    quote do  
      def log(msg), do: IO.puts("LOG: #{msg}")  
    end  
  end  
end  
  
defmodule MyApp do  
  use LoggerBase  
end
```

Използване

```
MyApp.log("Hello")
```

Какво прави кода?

LoggerBase е модул с макрос `__using__ /1`

```
defmacro __using__(_opts) do
  quote do
    def log(msg), do: IO.puts("LOG: #{msg}")
  end
end
```

- `__using__ /1` е специален макрос, който се извиква автоматично, когато друг модул направи `use LoggerBase`.
- Вътре в `quote do ... end` се дефинира код, който ще се “вкара” в другия модул.
- В случая се дефинира функция `log/1`, която отпечатва съобщение с префикс "LOG: ".

use LoggerBase в MyApp

```
defmodule MyApp do
  use LoggerBase
end
```

- `use LoggerBase` казва на Elixir: „Вземи `LoggerBase.__using__ /1` и вмъкни резултата тук“.
- Макросът добавя функцията `log/1` в модула `MyApp`.

Резултатът след компилация `MyApp` е еквивалентен на:

```
defmodule MyApp do
  def log(msg), do: IO.puts("LOG: #{msg}")
end
```

Извикване:

```
MyApp.log("Hello World") # => LOG: Hello World
```

Обобщение на пример 8:

- `LoggerBase.__using__ /1` е шаблонен макрос, който добавя функции в друг модул.
- `use LoggerBase` вкарва кода от макроса в текущия модул (`MyApp`). Това е често използван механизъм в Elixir за споделяне на функционалност, без да се използва наследяване.
- Резултатът показва, че `use LoggerBase` “inject”-ва функцията `log/1` в модула `MyApp`.

Или с други думи

- `use` извиква `__using__ /1` макрос.
- Кодът се инжектира при компилация.
- Използва се внимателно, тъй като може да скрие логика (т.е. програмистът, който чете `MyApp`, няма да вижда директно функцията `log/1`, защото тя се добавя чрез макроса `__using__ /1`. За да разбере какво се случва, трябва да отвори модула `LoggerBase`.)

Извод:

- `use` може да „скрие“ логика, защото добавя код автоматично чрез макроси, без този код да се вижда директно в модула, който го използва.

2.3. Протоколи

Сравняване на Протокол и Behaviour

Behaviour дефинира “договор” – кои функции трябва да “имплементира” модулът. Ползва се основно за съвместимост с `callback`-и (напр. `GenServer`).

Пример 9: В този пример се декларира `callback` функция, която определя интерфейс за модули, използващи дадено поведение.

```
defmodule WorkerBehaviour do
  @callback work(any) :: any
end
```

callback е декларация на функция, която модулът, използващ дадено поведение (`behaviour`), е длъжен да реализира.

- `@callback work(any) :: any` указва, че всеки модул, който използва `@behaviour WorkerBehaviour`, трябва да имплементира функция `work/1`.

- any означава, че функцията може да приема аргумент от произволен тип и да връща произволен резултат.

Protocol - дефинира “полиморфно поведение” според типа на данните, а не само модул. Позволява една и съща функция да работи по различен начин за различни типове структури.

Пример 10:

1) Дефиниране на протокол

```
defprotocol Size do
  def size(data)
end
```

Протоколът Size дефинира функцията size/1, която ще се реализира за различни типове данни.

2) Имплементация с List

```
defimpl Size, for: List do
  def size(list), do: length(list)
end
```

За тип List функцията size/1 връща броя на елементите.

3) Имплементация с Map

```
defimpl Size, for: Map do
  def size(map), do: map_size(map)
end
```

За Map се използва map_size/1, която връща броя на ключовете.

4) Използване

```
Size.size([1,2,3])    # => 3
Size.size(%{a:1, b:2}) # => 2
```

Връзката на Behaviour и Protocol с наследяване

- Elixir няма наследяване – няма класове и наследяване на state или функции.
- Behaviour и Protocol не са наследяване, но покриват някои аспекти на ООП наследяване:
 - Behaviour е близък до интерфейс – задължава модул да “имплементира” функции.
 - Protocol е близък до полиморфен метод – избира реализация според типа на структурата. Протоколът е отделен механизъм за постигане на полиморфизъм и поведение, паралелен на behaviour.

2.4. „Наследяване“ чрез behaviour (интерфейс)

Има ли интерфейс в Elixir?

В Elixir ролята на „интерфейс“ се изпълнява от behaviour.

Какво е дефакто behaviour?

behaviour е модул, който:

- дефинира callback функции,
- описва задължителен API за други модули.

Пример 11:

```
defmodule Worker do
  @callback work(any()) :: any()
end

defmodule Employee do
  @behaviour Worker

  def work(task) do
    "Работя по #{task}"
  end
end
```

Основните правила за behaviour в Elixir.

- Всички callback функции задължително се имплементират – модулът, който декларира @behaviour, трябва да реализира всички callback функции, иначе компилаторът ще даде предупреждение (warning).
- Осигурява договор, не код – @callback само описва интерфейс (функции и тяхната сигнатура), без да предоставя реална имплементация.
- Използва се за архитектурна яснота – поведението показва на други разработчици кои функции се очаква да бъдат реализирани, улеснявайки дизайна и поддръжката на системата.

Пример 12: Да се реализира интерфейсен механизъм в Elixir чрез behaviour, който дефинира функция за плащане, и да се демонстрира полиморфно извикване с различни имплементации.

1. Дефиниране на интерфейс (behaviour)

```
defmodule Payment do
  @callback pay(amount :: number()) :: :ok | :error
end
```

Този модул дефинира интерфейс (договор), който изисква всяка имплементация да има функцията pay/1.

2. Имплементация – плащане с карта

```
defmodule CardPayment do
  @behaviour Payment

  def pay(amount) do
    IO.puts("Плащане с карта: #{amount}")
    :ok
  end
end
```

Модулът декларира, че следва Payment behaviour и реализира задължителната функция pay/1.

3. Имплементация – плащане в брой

```
defmodule CashPayment do
  @behaviour Payment

  def pay(amount) do
    IO.puts("Плащане в брой: #{amount}")
    :ok
  end
end
```

Това е втора имплементация на същия интерфейс с различно поведение.

4. Полиморфна функция за обработка на плащане

```
def process(payment_module, amount) do
  payment_module.pay(amount)
end
```

Функцията приема модул като аргумент и извиква неговата функция pay/1, без да знае конкретната реализация.

5. Извикване на функцията с различни имплементации

```
process(CardPayment, 100)
```

Извиква се функцията pay/1 от модула CardPayment.

```
process(CashPayment, 50)
```

Извиква се функцията pay/1 от модула CashPayment.

Задачата включва:

- Behaviour, който играе ролята на интерфейс
- различни модули го имплементират
- функция process/2, която работи полиморфно
- изборът на поведение, осъществен чрез подаване на модул

Следователно: Не се интересуваме кой модул е подаден, важно е да се спазва интерфейса. Т.е. в Elixir няма класически интерфейси. *behaviour* изпълнява ролята на интерфейс. Той дефинира какви функции трябва да има, но не и как работят те. *behaviour* осигурява полиморфизъм и архитектурна яснота.

Използва ли се полиморфизъм в рамките на Elixir? Как се прилага в програмите?

Полиморфизмът в Elixir означава, че различни модули или типове данни могат да се използват по един и същ начин, стига да спазват един и същ договор (API). Това позволява кодът да бъде гъвкав, разширяем и независим от конкретни реализации.

Пример 14: Да се демонстрират различни видове полиморфизъм в Elixir чрез реализиране на поведение (*behaviour*), протокол, *pattern matching* и композиция чрез функции. За целта:

- 1) Да се дефинира *behaviour*, който описва общ интерфейс, и се създават няколко модула, които го имплементират.
- 2) Да се реализира *протокол* с различни реализации за различни типове данни.
- 3) Да се демонстрира функционален *полиморфизъм* чрез *pattern matching* и предпазни клаузи.
- 4) Да се използва *композиция*, като функция се подава като аргумент за постигане на различно поведение.
- 5) Да се покаже как Elixir постига полиморфизъм без класическо наследяване.

2.4.1. Полиморфизъм чрез *behaviour* (интерфейсен полиморфизъм)

Различни модули имплементират един и същ интерфейс (*behaviour*) и могат да се използват по един и същ начин.

```
defmodule Notifier do
  @callback send(msg :: String.t()) :: :ok
end
```

Този модул дефинира интерфейс, който изисква функция *send/1*.

```
defmodule EmailNotifier do
  @behaviour Notifier
  def send(msg), do: IO.puts("Email: #{msg}")
end
```

Модулът декларира, че следва интерфейса и реализира изпращане на съобщение по email.

```
defmodule SmsNotifier do
  @behaviour Notifier
  def send(msg), do: IO.puts("SMS: #{msg}")
end
```

Втора имплементация на същия интерфейс с различна логика.

```
def notify(module, message) do
  module.send(message)
end
```

Функцията приема модул като аргумент и извиква неговата функция *send/1*, без да знае конкретната реализация.

2.4.2. Полиморфизъм чрез *Protocols* (типово-ориентиран)

Една и съща функция има различна реализация в зависимост от типа на данните.

```
defprotocol Size do
  def size(data)
end
```

Дефинира се протокол, който описва функция *size/1*.

```
defimpl Size, for: List do
  def size(list), do: length(list)
end
```

Реализация на протокола за списъци.

```
defimpl Size, for: Map do
  def size(map), do: map_size(map)
end
```

Реализация на същия протокол за map тип.

```
Size.size([1,2,3])
Size.size(%{a: 1, b: 2})
```

Извиква се една и съща функция, но се изпълнява различен код според типа на аргумента.

2.4.3. Полиморфизъм чрез pattern matching (функционален полиморфизъм)

Една функция има няколко дефиниции, които се избират според формата или стойността на входните данни.

Какво е pattern matching?

Pattern matching е механизъм, при който структурата и стойността на данните се сравняват с шаблон, за да се:

- избере подходяща функция,
- извлекат стойности,
- контролира логиката на програмата.
- В Elixir = не означава присвояване, а съпоставяне на шаблон.

Основен синтаксис

```
pattern = value
```

Ако value съвпада с pattern, съпоставянето е успешно.

Примери за използване

1. Pattern matching при променливи

```
{x, y} = {1, 2}
```

x получава 1, а y получава 2.

2. Pattern matching в дефиниции на функции

```
def sum([], do: 0)
def sum([head | tail], do: head + sum(tail))
```

Elixir избира правилната функция според формата на аргумента.

3. Pattern matching със списъци

```
[head | tail] = [1, 2, 3]
```

head е 1, tail е [2, 3].

4. Pattern matching с map

```
%{name: name} = %{name: "Ana", age: 20}
```

Извлича се стойността на ключ :name.

5. Wildcard (_)

```
def process( _ ), do: :ok
```

_ приема всяка стойност, без да я запазва.

Guard clauses (предпазни клаузи)

```
def check(n) when n > 0, do: :positive
def check(n) when n < 0, do: :negative
def check( _ ), do: :zero
```

Guard-овете добавят допълнително условие към шаблона.

Основни правила на Pattern Matching

- Съпоставянето е отляво надясно.
- Шаблоните се проверяват в реда на дефиниране.
- Ако няма съвпадение, то възниква грешка.
- _ съвпада с всичко, но не пази стойност.
- Един шаблон не може да променя вече свързана променлива.

- Pattern matching работи със структури: списъци, tuple-и, map-и.

Следователно: *Pattern matching* е механизъм за съпоставяне на структурата на данните със шаблон и извличане на стойности. В Elixir той се използва при дефиниране на функции, присвояване и контрол на потока.

Да се върнем на Полиморфизъм чрез pattern matching

Пример 15

```
def process(0), do: "нула"
def process(n) when n > 0, do: "положително"
def process( _), do: "друго"
```

При извикване Elixir избира подходящата дефиниция на функцията според аргумента, както и guard условията.

Забележка:

*Редът **def process(_), do: "друго"** дефинира общ случай, който се изпълнява, когато входният аргумент не съвпада с останалите шаблони. т.е.:*

- `_` е wildcard (универсален шаблон), който означава „приеми каквато и да е стойност“.
- Този вариант на функцията се изпълнява, ако нито една от по-горните дефиниции не съвпадне.
- Така се гарантира, че функцията винаги ще върне резултат и няма да възникне грешка.

2.4.4. Полиморфизъм чрез data и functions (композиция)

Поведението се определя чрез подаване на функция като аргумент.

```
def apply_operation(value, operation) do
  operation.(value)
end
```

Функцията приема друга функция като параметър и я прилага върху подадената стойност, което позволява различно поведение без промяна на кода.

В контекста на Еликсир и функционалното програмиране термините “data” и “functions” при полиморфизъм чрез композиция (data и functions) се отнасят до това как се моделират обекти/структури с поведение без използване на класове и наследяване (т.е. без класическия ООП-подход).

1) Основната идея

В традиционното ООП полиморфизмът идва чрез наследяване и интерфейси

Задача: Създайте абстрактен клас Animal с чист виртуален метод: `virtual std::string speak() const = 0;` Този метод да се използва за извикване на “говоренето” на животните. Създайте два класа, които наследяват Animal:

- Dog – методът `speak()` трябва да връща "Woof".
- Cat – методът `speak()` трябва да връща "Meow".

В главната функция (`main()`):

- Създайте колекция (например `std::vector<std::unique_ptr<Animal>>`) от различни животни (Dog и Cat).
- Извикайте полиморфно метода `speak()` за всяко животно и отпечатайте резултата.
- Използвайте виртуален деструктор в Animal за унищожаване на обектите.

Примерна реализация в C++

```
#include <iostream>
#include <vector>
#include <memory>                                // за smart pointers

// Базов клас Animal
class Animal {
public:
  virtual std::string speak() const = 0;          // чист виртуален метод (абстрактен)
```

```

virtual ~Animal() {} // виртуален деструктор
};

// Клас Dog наследява Animal
class Dog : public Animal {
public:
    std::string speak() const override {
        return "Woof";
    }
};

// Клас Cat наследява Animal
class Cat : public Animal {
public:
    std::string speak() const override {
        return "Meow";
    }
};

int main() {
    // Създаваме колекция от животни чрез pointers
    std::vector<std::unique_ptr<Animal>> animals;
    animals.push_back(std::make_unique<Dog>());
    animals.push_back(std::make_unique<Cat>());

    // Полиморфно извикване на speak
    for (const auto& animal : animals) {
        std::cout << animal->speak() << std::endl;
    }
    return 0;
}

```

Тъй като в Еликсир нямаме класове, за да постигнем полиморфизъм чрез композиция използваме комбиниране на data и functions за създаване на структури, които имат различно поведение.

2) Data (данни)

data е структура, която държи състояние. В Elixir това може да е **struct** или **map**:

```

defmodule Dog do
  defstruct name: "", breed: ""
end

dog = %Dog{name: "Rex", breed: "Shepherd"}

```

Това е само съхранение на данни без логика.

3) Functions (функции)

functions са операциите върху данните. В полиморфизма чрез композиция, вместо да наследяваме методи ние свързваме функции с конкретни данни. Тук имаме отделни функции за различните “типове животни”:

```

def dog_speak(dog), do: "#{dog.name} says Woof"
def cat_speak(cat), do: "#{cat.name} says Meow"

```

4) Комбиниране на data и functions (composition)

Идеята на полиморфизма чрез композиция е да създадем обекти като map, които съдържат и състояние и поведение:

```

defmodule Animal do
  # Функция за създаване на "Dog" обект
  def dog(name) do
    %{
      type: :dog,

```

```

    name: name,
    speak: fn -> "#{name} says Woof" end
  }
end

# Функция за създаване на "Cat" обект
def cat(name) do
  %{
    type: :cat,
    name: name,
    speak: fn -> "#{name} says Meow" end
  }
end

# Създаваме колекция от животни
animals = [Animal.dog("Rex"), Animal.cat("Misty")]

# Полиморфно извикване на speak
Enum.each(animals, fn animal ->
  IO.puts(animal.speak()) # Всяко животно "говори" чрез прилагане на съответната функция
end)

```

```

data: %{type: :dog, name: name}
functions: speak: fn -> ... end

```

Това дефакто извикване е полиморфизъм: всеки "животински обект" има една и съща структура и API (speak), но различна реализация. Това е функционален еквивалент на интерфейс/полиморфизъм от ООП.

Едно и също извикване на функция може да доведе до различно поведение в зависимост от обекта. В конкретния код извикването е:

```
animal.speak()
```

Това извикване е еднакво за всички животни, но резултатът е различен:

```

Rex says Woof
Misty says Meow

```

Причината е, че всяко животно има различна функция speak вътре в тар-а.

Принцип на работа при полиморфизъм в Еликсир

Animal.dog("Rex"), връща:

```

%{
  type: :dog,
  name: "Rex",
  speak: fn -> "Rex says Woof" end
}

```

и

```
Animal.cat("Misty")
```

което от своя страна връща:

```

%{
  type: :cat,
  name: "Misty",
  speak: fn -> "Misty says Meow" end
}

```

След това се изпълнява Enum.each

```

Enum.each(animals, fn animal ->
  IO.puts(animal.speak())
end)

```

тук не ни интересува какъв тип е `animal`, а извикваме `animal.speak()` и всяко животно изпълнява своята версия на `speak`.

В Elixir се използват модулни псевдоними (**alias**) и директни функции (**import**). Предназначението на **alias** е да съкрати името на модул и да се улесни писането на кода, като вместо пълно име можем да използваме кратък псевдоним. **import** се използва, за да извикваме функции или макроси от друг модул директно, без да пишем името на модула пред тях.

A. alias (псевдоним)

- `alias` се използва за съкратено изписване на модулни имена.
- Често модулите имат дълги имена (например `MyApp.Accounts.User`).
- С `alias` можем да дадем кратко име, което ще използваме в кода.

Синтаксис

```
alias Full.Module.Name  
# или с псевдоним  
alias Full.Module.Name, as: ShortName
```

Пример 16

```
defmodule Example do  
  # Дългото име MyApp.Accounts.User можем да съкратим с alias  
  alias MyApp.Accounts.User  
  def create_user(name) do  
    # вместо да пишем MyApp.Accounts.User.new(name)  
    User.new(name)  
  end  
end
```

Без alias:

```
MyApp.Accounts.User.new("Alice")
```

С alias:

```
User.new("Alice")
```

Друг пример с псевдоним:

```
alias MyApp.Accounts.User, as: Person  
Person.new("Bob")
```

Предимства от употребата

- Прави кода по-кратък и четим
- Избягва дълги квалифицирани имена

B. import

`import` се използва за директно използване на функции или макроси от друг модул, без да се пише името на модула.

Синтаксис

```
import ModuleName  
# или само конкретни функции  
import ModuleName, only: [function_name: arity]  
# или изключваме функции  
import ModuleName, except: [function_name: arity]
```

Пример 17

```
defmodule Example do  
  # Импортираме всички функции от модул Enum  
  import Enum  
  
  def demo do  
    list = [1, 2, 3]
```

```
# вместо Enum.map(list, &(&1 * 2))
map(list, &(&1 * 2))
end
end
```

С `import` можем да извикаме функцията директно, без `Enum`.

Пример с ограничаване на функции:

```
defmodule Example2 do
  import Enum, only: [map: 2]

  def demo do
    map([1, 2, 3], &(&1 + 1))
    # filter([1, 2, 3], &(&1 > 1)) -> това ще даде грешка, защото не е импортирано
  end
end
```

Предимства:

- Позволява по-кратки изрази, особено когато извикваме често функции от даден модул
- Можем да изберем точно кои функции да импортираме (`only`) или кои да изключим (`except`)

Таблица 4. Сравнение `alias` с `import`

Функция	Какво прави	Пример
<code>alias</code>	Съкратено име на модул	<code>alias MyApp.Accounts.User;</code> <code>User.new()</code>
<code>import</code>	Позволява извикване на функции директно	<code>import Enum; map([1,2,3],</code> <code>&(&1*2))</code>

Основна разлика:

- `alias` съкращава името на модула
- `import` съкращава извикването на функции или макроси от модул

Пример за комбинирано използване:

```
defmodule MyApp do
  # Съкратено име на модул
  alias MyApp.Accounts.User

  # Импортираме директно map функцията
  import Enum, only: [map: 2]

  def demo do
    users = [%User{name: "Alice"}, %User{name: "Bob"}]

    # map директно без Enum.
    map(users, fn user -> user.name end)
  end
end
```

User чрез `alias`, `map` чрез `import`

Обобщение

Elixir няма класическо наследяване, както в C++ или Ruby.

1. “Позволява разширяемост, без да е нужно наследяване”.

- В Elixir разширяемостта се постига чрез създаване на нови модули или структури (`map/struct`) и функции, вместо чрез наследяване. Това е ключова особеност, защото не се нуждаем от базови класове или виртуални методи.

2. “Поддържа явна композиция на функции и данни”

- Полиморфизмът се постига чрез свързване на `data` + функции в един обект (`map/struct`). Всяко “животно” има същия интерфейс (например `:speak`), но различна реализация. Това е чистият Elixir начин за полиморфизъм, без класове.

3. “Работи перфектно с `immutable data`”

- Всички структури в Elixir са *immutable*, т.е. състоянието не се променя. Това елиминира проблеми с непреднамерени странични ефекти, типични за *mutable* обекти в класическите езици.
4. “Лесно се подменя поведението (*swap функции*) по време на *runtime*”
- Тъй като функциите са първокласни обекти, можем да ги присвояваме, променяме или подменяме в *runtime*. Това е еквивалент на *override* на методи, но без наследяване.
5. Как се прилага *Полиморфизмът в реални програми*?
- Създаване на плъгини (различни имплементации).
 - Работа с външни услуги (API клиенти).
 - Тестване чрез *mock имплементации*.
 - Писане на гъвкав и разширяем код.
6. По какъв начин използването на интерфейси е свързано с полиморфизъм?
- Интерфейсите са механизъм, чрез който се реализира полиморфизъм. Те позволяват един и същ код да работи с различни реализации, без да знае какви са те. В контекста на Elixir
- В Elixir няма класически интерфейси.
 - Ролята на интерфейс се изпълнява от *behaviour*, който *behaviour* дефинира какви функции трябва да има един модул. Различни модули могат да имплементират този *behaviour*.
- Това е полиморфизъм:
- един интерфейс който има много реализации.

Пример 18

```
def process(module, data) do
  module.handle(data)
end
```

Тук *process/2*:

- не знае кой е модулът,
- важно е само да има *handle/1*.

С други думи интерфейсите (в Elixir – *behaviour*) са свързани с полиморфизма, защото позволяват различни реализации да се използват по един и същ начин. Можем да направим прилика с интерфейсът е като контакт, а различните реализации са различните уреди – контактът е един, уредите са много.

Таблица 5. Сравнение между композиция, *use* с макроси, *protocol*, *behaviour* и *import / alias*

Механизъм	Какво осигурява?
Композиция	Съчетаване на данни и функции (<i>data + functions</i>) за постигане на полиморфизъм
<i>use</i> с макроси	Включване на функционалност от друг модул; повторна употреба на код
Протоколи (<i>protocol</i>)	Дефинират поведение (интерфейс) за различни типове данни; полиморфно поведение
Поведение (<i>behaviour</i>)	Дефинира задължителни функции, които модулът трябва да имплементира; подобно на интерфейс в ООП
<i>import / alias</i>	Улеснява достъпа до функции и модули; съкращава имена и подпомага композицията

- **Компиляция и скриптов режим** – Elixir кодът може да се компилира или да се изпълнява като скрипт.
- **Наименувани функции** – дефинират се с *def* и принадлежат на модул.
- **Function capturing** – функции могат да се третират като стойности.
- **Аргументи по подразбиране** – улесняват извикването на функции.

1) Само *d* с подразбираща се стойност (последен аргумент)

```
def f(a, b, c, d \\ 0), do: {a, b, c, d}

f(1,2,3)    # => {1,2,3,0}
f(1,2,3,4)  # => {1,2,3,4}
```

Допустимо, защото *d* е на края.

2) с и *d* с подразбираща се стойност

```
def f(a, b, c \ 0, d \ 0), do: {a, b, c, d}

f(1,2)      # => {1,2,0,0}
f(1,2,3)    # => {1,2,3,0}
f(1,2,3,4)  # => {1,2,3,4}
```

Допустимо, защото всички с подразбиращи се стойности са в края.

3) b, c, d с подразбираща се стойност

```
def f(a, b \ 0, c \ 0, d \ 0), do: {a, b, c, d}

f(1)        # => {1,0,0,0}
f(1,2)      # => {1,2,0,0}
f(1,2,3)    # => {1,2,3,0}
f(1,2,3,4)  # => {1,2,3,4}
```

Допустимо, а е задължителен, останалите са optional.

4) Всички с подразбираща се стойност

```
def f(a \ 0, b \ 0, c \ 0, d \ 0), do: {a, b, c, d}

f()          # => {0,0,0,0}
f(1)         # => {1,0,0,0}
f(1,2,3,4)   # => {1,2,3,4}
```

Допустимо, всички аргументи са optional.

5) a и c с подразбираща се стойност

```
def f(a \ 0, b, c \ 0, d), do: {a, b, c, d} # Грешка
```

НЕДОПУСТИМО: аргумент с подразбираща се стойност (a) е пред аргумент без подразбираща се стойност (b), същото важи за c пред d.

Компиляторът дава синтактична грешка.

Правилно решение, ако искате a и c с подразбираща се стойност

```
def f(b, d, a \ 0, c \ 0), do: {a, b, c, d}
```

```
f(1,2)      # => {0,1,0,2}
```

- 0 е ст-стта за a, намира се на 3 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се в началото {0,1,0,2}

- 1 е ст-стта за b, намира се на 1 място в списъка с с подразбиращи се стойности f(b, d, a \ 0, c \ 0), отпечатва се на 2 място {0,1,0,2}

- 0 е ст-стта за c, намира се на 4 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се на 3 място {0,1,0,2}

- 2 е ст-стта за d, намира се на 2 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се на 4 място {0,1,0,2}

.....

```
f(5,6,7,8)  # => {7,5,8,6}
```

Ако не използваме стойностите по подразбиране

- 5 е ст-стта за b, намира се на 1 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се на 2 място {7,5,8,6}

- 6 е ст-стта за d, намира се на 2 място в списъка с с подразбиращи се стойности f(b, d, a \ 0, c \ 0), отпечатва се на 4 място {7,5,8,6}

- 7 е ст-стта за a, намира се на 3 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се на 1 място {7,5,8,6}

- 8 е ст-стта за c, намира се на 4 място в списъка с подразбиращи се стойности f(b, d, a \ 0, c \ 0) отпечатва се на 3 място {7,5,8,6}

Тук всички аргументи с подразбираща се стойност са в края. Технически е валидно, но редът на резултата е различен от естествения (**а и с идват първо в кортежа**, но се задават на края при извикване).

Обобщение – модулите подобряват организацията и повторната употреба на кода.

3. Функции (разгледано в лекция 2 и по-горе в настоящата лекция)

Функциите са основният механизъм за реализиране на логика.

- **Анонимни функции** – дефинират се с `fn ... end` и се използват често като аргументи.
- **Capture оператор** – съкратен синтаксис за анонимни функции.
- **Pattern Matching** – използва се при дефиниране и извикване на функции.
- **Наименувани и частни функции** – публични (`def`) и частни (`defp`).
- **Аргументи по подразбиране** – позволяват гъвкави извиквания.

Обобщение – функциите са чисти, изразителни и лесни за тестване.

4. Модули и функции – допълване

- **Съставяне на модул и тяло на функция** (разгледано в предходна лекция 2)
- **Извикване на функции и pattern matching**

В Elixir функциите могат да имат няколко дефиниции, като правилната се избира чрез pattern matching върху аргументите.

Пример 19

```
defmodule Math do
  def sign(0) do
    "нула"
  end

  def sign(n) when n > 0 do
    "положително"
  end

  def sign(n) do
    "отрицателно"
  end
end
```

- `sign(0)` – съвпада само ако аргументът е 0.
- `sign(n) when n > 0` – използва `guard`, за да провери дали числото е положително.
- последната функция обработва всички останали случаи.

Извикване

```
Math.sign(0) # => "нула"
Math.sign(5) # => "положително"
Math.sign(-3) # => "отрицателно"
```

Elixir избира първата дефиниция, която съвпада с аргументите.

В Elixir операторът `=` не е класическо присвояване, а съпоставяне на шаблон (pattern).

Пример 20

```
{x, y} = {10, 20}
```

Резултат

```
x = 10
y = 20
```

Шаблонът `{x, y}` съвпада със структурата `{10,20}`.

Несъвпадение

```
{x, y} = {10, 20, 30}
```

Резултат

```
MatchError
```

Причината е, че структурата не съвпада.

2. Pattern Matching в функции

Може да се дефинират различни версии на една функция.

Пример 21

```
defmodule Example do
  def greet("Ivan") do
    "Hello Ivan!"
  end

  def greet(name) do
    "Hello #{name}"
  end
end
```

Извикване

```
Example.greet("Ivan") # Hello Ivan!
Example.greet("Maria") # Hello Maria
```

Elixir избира първата функция, която съвпада.

3. Pattern Matching със списъци

Списъците могат да се разделят на глава (head) и опашка (tail).

Пример 22

```
[head | tail] = [1,2,3,4]
```

Резултат

```
head = 1
tail = [2,3,4]
```

Пример 23 с функция

```
defmodule ListExample do
  def first([h | _]) do
    h
  end
end
```

Извикване

```
ListExample.first([10,20,30])
# => 10
```

4. Pattern Matching с кортежи

Пример 24

```
{:ok, result} = {:ok, 100}
```

Резултат

```
result = 100
```

Пример 25 при работа с файлове

```
case File.read("test.txt") do
  {:ok, content} -> IO.puts(content)
  {:error, reason} -> IO.puts("Error: #{reason}")
end
```

Elixir проверява кой шаблон съвпада.

5. Pattern Matching с Map

Пример 26

```
person = %{name: "Ivan", age: 30}
%{name: name} = person
```

Резултат

```
name = "Ivan"
```

Не е нужно да се описват всички ключове.

6. Pattern Matching в параметри на функция

```
defmodule User do
  def show(%{name: name}) do
    "User name: #{name}"
  end
end
```

Извикване

```
User.show(%{name: "Ivan", age: 30})
# => "User name: Ivan"
```

7. Pattern Matching със списък от точно два елемента

```
[a,b] = [5,10]
```

Резултат

```
a = 5
b = 10
```

8. Игнориране на стойност

Символът `_` означава игнорирана стойност.

```
{_, y} = {10,20}
```

Резултат

```
y = 20
```

Кога се прилага ?

- Игнориране на елемент от tuple

Когато функция връща tuple, но ни трябва само една стойност.

Пример:

```
{number, _} = Float.parse("12.5abc")
```

- number ще бъде 12.5
- _ игнорира "abc"

- Игнориране на параметър във функция

Ако функцията получава аргумент, който не използваме, можем да го заменим с `_`.

Пример 27

```
def print_first(a, _) do
  IO.puts(a)
end
```

Извикване

```
print_first(5, 10)
```

- Ще се изведе: 5
- Втората стойност се игнорира.

- Игнориране в pattern matching със списъци

Когато искаме да вземем само първия елемент.

Пример 28

```
[a, _, _] = [1, 2, 3]
```

Резултат

```
a = 1
```

Вторият и третият елемент се игнорират.

Правилото за игнориране гласи: „тук има стойност, но ние не се интересуваме каква е тя“!

Примери на Elixir, използващи функционален стил на програмиране

1. Да се създаде приложение Квадратно уравнение от вида $a \cdot x^2 + b \cdot x + c = 0$. За целта да се използва формула (2.1). Да се пресметнат дискриминантата и квадратните корени на уравнението по зададени стойности за a, b и c. Да се направи проверка за коректни входни данни, да се изведат подходящи съобщения, ако уравнението има два еднакви корена или няма реални корени. Резултата да се закръгли до втори знак след десетичната запетая.

$$x_{1,2} = (-b \pm \sqrt{b^2 - 4 \cdot a \cdot c}) / (2 \cdot a) \quad (1.1)$$

Решение 1:

```
defmodule T do
  def test(a, b, c) do
    if String.match?(a, ~r/^\d+(\.\d{1,2})?$/) and String.match?(b, ~r/^\d+(\.\d{1,2})?$/) and String.match?(c, ~r/^\d+(\.\d{1,2})?$/) do
      {a, _} = Float.parse(a)
      {b, _} = Float.parse(b)
      {c, _} = Float.parse(c)
      d = b * b - 4 * a * c
      IO.puts("D is: #{d}")
      if d < 0 do
        IO.puts("No real roots!")
      else
        if d == 0 do
          x1 = (-b / (2 * a)) >> Float.round(2)
          IO.puts("X1=X2 is: #{x1}")
        else
          if d > 0 do
            # d = :math.sqrt(d)
            x1 = (-b + :math.sqrt(d) / (2 * a)) >> Float.round(2)
            x2 = (-b - :math.sqrt(d) / (2 * a)) >> Float.round(2)
            IO.puts("X1 is: #{x1}")
            IO.puts("X2 is: #{x2}")
          end
        end
      end
    else
      IO.puts("One of the variables is not a number")
    end
  end
end
```

```
a = IO.gets("Enter a: ") >> String.trim()
b = IO.gets("Enter b: ") >> String.trim()
c = IO.gets("Enter c: ") >> String.trim()
IO.puts(T.test(a, b, c))
```

Вход:

```
Enter a: 2
Enter b: 4
Enter c: 2
D is: 0.0
X1=X2 is: -1.0
ok
....
Enter a: 2
Enter b: 6
Enter c: 4
D is: 4.0
X1 is: -5.5
X2 is: -6.5
ok
....
Enter a: 4
Enter b: 1
Enter c: 6
D is: -95.0
No real roots!
ok
```

Постъпково пояснение

Ред	Код	Функция / Какво прави
1	defmodule T do	Декларира модул с име T. В Elixir всички функции трябва да са в модул.
2	def test(a, b, c) do	Дефинира функция test, която приема три аргумента a, b и c.
3	if String.match?(a, ~r/^\d+(\.\d{1,2})?\$/) and String.match?(b, ~r/^\d+(\.\d{1,2})?\$/) and String.match?(c, ~r/^\d+(\.\d{1,2})?\$/) do	Проверява дали a, b и c са числа във формат „цяло или десетично с максимум 2 цифри след десетичната точка“. Използва регулярни изрази.
4	{a, _} = Float.parse(a)	Парсава стринга a в число с плаваща запетая и присвоява резултата обратно на a. Частта _ игнорира остатъка от парсинга.
5	{b, _} = Float.parse(b)	Същото за b – преобразува го в Float.
6	{c, _} = Float.parse(c)	Същото за c.
7	d = b * b - 4 * a * c	Изчислява дискриминантата D на квадратното уравнение ($ax^2 + bx + c = 0$).
8	IO.puts("D is: #{d}")	Показва стойността на дискриминантата в конзолата.
9	if d < 0 do	Проверява дали дискриминантата е отрицателна.
10	IO.puts("No real roots!")	Ако d < 0, извежда, че няма реални корени.
11	else	Алтернативен клон, ако d >= 0.
12	if d == 0 do	Проверява дали дискриминантата е нула (единичен реален корен).
13	x1 = (-b / (2 * a)) >> Float.round(2)	Извършва изчислението и закръгля до 2 знака след десетичната запетая
14	IO.puts("X1=X2 is: #{x1}")	Извежда корена, който е един и същ за X1 и X2.
15	else	Алтернативен клон, ако d > 0 (две различни реални решения).
16	if d > 0 do	Проверява, че дискриминантата е положителна.
17	x1 = (-b + :math.sqrt(d) / (2 * a)) >> Float.round(2)	Извършва изчислението и закръгля до 2 знака след десетичната запетая
18	x2 = (-b - :math.sqrt(d) / (2 * a)) >> Float.round(2)	Извършва изчислението и закръгля до 2 знака след десетичната запетая
19	IO.puts("X1 is: #{x1}")	Извежда първия корен.
20	IO.puts("X2 is: #{x2}")	Извежда втория корен.
21	end	Край на if d > 0.
22	end	Край на if d == 0.
23	end	Край на if d < 0.
24	else	Алтернативен клон на първата проверка – ако някой от входовете не е число.
25	IO.puts("One of the variables is not a number")	Извежда съобщение, че поне една от променливите не е валидно число.
26	end	Край на първия if.
27	end	Край на функцията test.
28	end	Край на модула T.
1	a = IO.gets("Enter a: ") >> String.trim()	Показва съобщението "Enter a: " на потребителя. Програмата спира и чака потребителя да въведе нещо в конзолата. Резултатът е низ, който включва въведения текст + символа за нов ред (\n). След това имаме т.н. >> Pipe операторът, подава резултата на IO.gets към String.trim, който премахва ненужните празни символи от началото и края на даден низ.
2	b = IO.gets("Enter b: ") >> String.trim()	Извиква функцията test от модул T, като ѝ подава трите променливи a, b и c. След това резултатът се отпечата на конзолата чрез IO.puts.
3	c = IO.gets("Enter c: ") >> String.trim()	
4	IO.puts(T.test(a, b, c))	

2. Да се разработи приложение Температурен конвертор за преобразуване на температура от Целзий във Фаренхайт и обратно. Да се направят подходящи промени в свойствата на контролата за извеждане на резултата. Да се закръгли резултата до втори знак след десетичната запетая. За целта да се приложат формула (3.1) и (3.2).

$$F = ((C * 9) / 5) + 32 \quad (2.1)$$

$$C = ((F - 32) * 5) / 9 \quad (2.2)$$

Решение 1:

```
defmodule T do
  def conv(temp, convo) do
    if String.match?(temp, ~r/^\d+(\.\d{1,2})?$/) and String.match?(convo, ~r/^\d+(\.\d{1,2})?$/) do
      {temp, _} = Float.parse(temp)
      {convo, _} = Integer.parse(convo)
      if convo == 1 do
        f = (((temp * 9) / 5) + 32) |> Float.ceil(2)
        IO.puts("#{temp} C is #{f} F")
      else
        if convo == 2 do
          c = (((temp - 32) * 5) / 9) |> Float.ceil(2)
          IO.puts("#{temp} F is #{c} C")
        else
          IO.puts("You have entered an invalid mode!")
        end
      end
    end
    else
      IO.puts("One of the entered values is not a number")
    end
  end
end
IO.puts("Enter temperature and conversion (1. C -> F 2. F -> C)")
temp = IO.gets("Temperature: ") |> String.trim()
convo = IO.gets("Mode: ") |> String.trim()
IO.puts(T.conv(temp, convo))
```

Ред	Код	Обяснение
1	defmodule T do	Дефинира модул с име T. Всички функции вътре в този блок ще принадлежат на този модул.
2	def conv(temp, convo) do	Дефинира функция conv/2 в модула T, която приема два аргумента: temp (температура) и convo (конверсионен режим).
3	if String.match?(temp, ~r/^\d+(\.\d{1,2})?\$/) and String.match?(convo, ~r/^\d+(\.\d{1,2})?\$/) do	Проверява дали: 1) temp е число (цяло или десетично с до 2 цифри след десетичната точка) и 2) convo е цяло число. Ако и двете са верни, се изпълнява следващият блок; иначе – блокът след else.
4	{temp, _} = Float.parse(temp)	Преобразува низа temp в число с плаваща запетая (Float). Игнорира втория елемент от кортежа (неизползван).
5	{convo, _} = Integer.parse(convo)	Преобразува низа convo в цяло число (Integer).
6	if convo == 1 do	Проверява дали режимът е 1, т.е. конверсия Celsius → Fahrenheit.
7	f = (((temp * 9) / 5) + 32) > Float.ceil(2)	Извършва изчислението и закръгля до 2 знака след десетичната запетая.
8	IO.puts("#{temp} C is #{f} F")	Извежда резултата на конзолата, напр. "25 C is 77 F".
9	else	Ако convo не е 1, проверява следващия вариант.
10	if convo == 2 do	Проверява дали режимът е 2, т.е. конверсия Fahrenheit → Celsius.
11	c = (((temp - 32) * 5) / 9) > Float.ceil(2)	Извършва изчислението и закръгля до 2 знака след десетичната запетая.
12	IO.puts("#{temp} F is #{c} C")	Извежда резултата на конзолата, напр. "77 F is 25 C".
13	else	Ако convo не е нито 1, нито 2, изпълнява този блок.
14	IO.puts("You have entered an invalid mode!")	Извежда съобщение за грешен режим.
15	end	Край на вътрешния if за режима.
16	else	Ако проверката за числата (temp и convo) не е вярна, се изпълнява този блок.
17	IO.puts("One of the entered values is not a number")	Извежда съобщение за грешен вход, ако едно от въведените значения не е число.
18	end	Край на външния if.
19	end	Край на функцията conv/2.
20	end	Край на модула T.
21	IO.puts("Enter temperature and conversion (1. C -> F 2. F -> C)")	Извежда инструкции за потребителя какво да въведе.
22	temp = IO.gets("Temperature: ") > String.trim()	Показва съобщението "Enter a: " на потребителя. Програмата спира и чака потребителя да въведе нещо в конзолата. Резултатът е низ, който включва въведения текст + символа за нов ред (\n). След това имаме т.н. > Pipe операторът, подава резултата на IO.gets към String.trim, който премахва ненужните празни символи от началото и края на даден низ.
23	convo = IO.gets("Mode: ") > String.trim()	Извиква функцията conv/2 от модул T с аргументите temp и convo и отпечатва резултата (или съобщението за грешка).
24	IO.puts(T.conv(temp, convo))	

В редовете:

```
{a, _} = Float.parse(a)
```

```
{b, _} = Float.parse(b)
```

```
{c, _} = Float.parse(c)
```

Функцията Float.parse/1 връща tuple с два елемента: {число, остатък}

Пример 29

```
Float.parse("12.5abc")
```

Резултат:

```
{12.5, "abc"}
```

- първият елемент е числото
- вторият елемент е останалата част от низа

9. Pattern Matching в case

```
x = 5

case x do
  0 -> "zero"
  5 -> "five"
  _ -> "other"
end
```

Резултат

```
"five"
```

● Предпазни клаузи (guards)

Предпазни клаузи (Guards)

Guards са допълнителни условия към pattern matching, които ограничават кога дадена функция може да бъде извикана. Използват се с ключовата дума: when

Пример 30

```
defmodule Numbers do
  def check(x) when is_integer(x) and x > 0 do
    "положително цяло число"
  end

  def check(x) when is_integer(x) and x < 0 do
    "отрицателно цяло число"
  end

  def check(_) do
    "друг тип стойност"
  end
end
```

- is_integer(x) проверява дали стойността е цяло число.
- and x > 0 добавя допълнително условие.
- _ означава „произволна стойност“.

Извикване

```
Numbers.check(10)      # => "положително цяло число"
Numbers.check(-2)      # => "отрицателно цяло число"
Numbers.check("abc")   # => "друг тип стойност"
```

Често използвани guard функции

- is_integer(x)
- is_float(x)
- is_number(x)
- is_list(x)
- is_atom(x)
- length(list)
- сравнения (>, <, ==)

Разлика между number и integer и float

number

- integer

- float

Пример 31

```
is_integer(10)      # true
is_integer(10.5)    # false

10                  # integer
10.5                # float
-3                  # integer
```

```

0.25                # float
is_number(10)       # true
is_number(10.5)     # true

x = 5
y = 5.2

IO.puts(is_number(x)) # true
IO.puts(is_integer(x)) # true
IO.puts(is_number(y)) # true
IO.puts(is_integer(y)) # false

```

Обобщение: integer включва само цели числа, а number включва всички числа, както integer така и float

● Частни функции и параметри по подразбиране

Частни функции

Частните функции се дефинират с: defp. Те могат да се използват само вътре в модула.

Пример 31

```

defmodule Calculator do
  def square_sum(a, b) do
    add(square(a), square(b))
  end
  defp square(x) do
    x * x
  end
  defp add(x, y) do
    x + y
  end
end

```

square/1 и add/2 са частни функции. Те се използват само в square_sum/2.

Извикване

```

Calculator.square_sum(2,3)
# => 13

```

Опит за извикване отвън - ще даде грешка, защото функцията е частна.

```

Calculator.square(3)

```

Параметри по подразбиране

Стойност по подразбиране се задава с оператора: \

Пример 32

```

defmodule Greeter do
  def hello(name \ "Guest") do
    IO.puts("Hello #{name}")
  end
end

```

Извикване

```

Greeter.hello("Ivan")
# Hello Ivan
Greeter.hello()
# Hello Guest

```

Пример 34 с повече параметри

```
defmodule Calc do
  def add(a, b \\ 0, c \\ 0) do
    a + b + c
  end
end
```

Извикване

```
Calc.add(5)      # 5
Calc.add(5,2)    # 7
Calc.add(5,2,3)  # 10
```

Правило: Всички параметри със стойности по подразбиране трябва да са в края на списъка с аргументи.

- **Директиви** – import, alias, require, use (*разгледано в настоящата лекцията по-горе*)
- **Имена на модули и използване на Erlang библиотеки** (*преговор с разширение*)

1. Имена на модули в Elixir

В Elixir модулите се именуват с главна буква (PascalCase) и могат да бъдат йерархично организирани чрез точка ..

Основни правила

- Името започва с главна буква.
- Използва се PascalCase (всяка дума започва с главна буква).
- Подмодулите се разделят с точка.
- Модул се дефинира с defmodule.

Пример 35

```
defmodule MathUtils do
  def add(a, b) do
    a + b
  end
end
```

Извикване:

```
MathUtils.add(2,3)
# => 5
```

MathUtils е модул, който съдържа функцията add/2.

Йерархични модули

```
defmodule MyApp.Math.Calculator do
  def multiply(a, b) do
    a * b
  end
end
```

Извикване:

```
MyApp.Math.Calculator.multiply(3,4)
# => 12
```

Това е йерархия от модули, използвана за по-добра организация на кода.

Съкратено използване на модул - разширение

Може да използваме alias, за да съкратим името.

```
alias MyApp.Math.Calculator
Calculator.multiply(2,5)
```

В случая директно е използвано името Calculator.

В този код новото име се дефинира в реда с `alias`.

```
alias MyApp.Math.Calculator
Calculator.multiply(2,5)
```

`alias MyApp.Math.Calculator` създава съкратено име (псевдоним) на модула. След това вместо пълното име `MyApp.Math.Calculator.multiply(2,5)` можем да използваме кратката форма `Calculator.multiply(2,5)`. Тоест **Calculator** е новото (съкратено) име, създадено чрез `alias`.

Причината да не се изписва съкратеното име изрично е, че Elixir автоматично използва последната част от името на модула като `alias`, ако не е зададено друго.

```
alias MyApp.Math.Calculator
```

Тук `Calculator` се извлича автоматично от пълното име `MyApp.Math.Calculator`.

Следователно: `Calculator.multiply(2,5)` е съкратена форма на: `MyApp.Math.Calculator.multiply(2,5)`

Ако искате да зададете различно име, трябва да го напишете изрично:

```
alias MyApp.Math.Calculator, as: Calc
Calc.multiply(2,5)
```

`as: Calc` задава ново име (псевдоним) на модула.

Важно правило за използване на `alias`!!!

- Без `as`: - използва се последната част от името на модула.
- С `as`: - използва се името, което зададете ръчно.

2. Използване на Erlang библиотеки

Elixir работи върху Erlang VM (BEAM), затова може директно да използва Erlang модули. Erlang модулите се разпознават по това, че започват с `:` (атом).

Пример 1 – математическа библиотека

```
:math.sqrt(9)
```

Резултат

```
3.0
```

`:math` е Erlang модул за математически функции.

Пример 2 – генериране на случайно число

```
:rand.uniform(10)
```

Резултат

```
число между 1 и 10
```

Пример 3 – работа с време

```
:calendar.local_time()
```

Резултат

```
{{2025,3,6},{14,30,10}}
```

Пример 4 – Erlang конверсия

```
:list_to_atom('hello')
```

Резултат

```
hello
```

Таблица 1. Разлика между Elixir и Erlang модули

Тип модул	Пример	Забележка
Elixir	<code>String.length("abc")</code>	започва с главна буква
Erlang	<code>:math.sqrt(9)</code>	започва с :

Съпоставка между модули в Elixir и Erlang библиотеки

Модули в Elixir

- започват с главна буква
- използват PascalCase
- могат да имат йерархия чрез точки

Erlang библиотеки

- използват се директно в Elixir
- модулите са атоми (:module)
- извикват се като :module.function()

Класификация на често използвани Erlang модули в Elixir

В Elixir, Erlang модулите се извикват с :module_name.

1. Модул :io – вход и изход

Използва се за четене и писане в конзолата.

Пример

```
defmodule IoExample do
  def run do
    # извеждане на текст
    :io.format("Hello from Erlang IO module~n")

    # извеждане на число
    :io.format("Number: ~p~n", [10])
  end
end
```

IoExample.run()

- ~n - нов ред
- ~p - отпечатва стойност
- вторият аргумент е списък с параметри

Изход

```
Hello from Erlang IO module
Number: 10
```

2. Модул :math – математически функции

Съдържа тригонометрични функции, коренуване, степенуване.

Пример

```
defmodule MathExample do
  def run do
    sqrt = :math.sqrt(25)
    pow = :math.pow(2, 3)

    IO.puts("Square root: #{sqrt}")
    IO.puts("Power: #{pow}")
  end
end
```

MathExample.run()

- :math.sqrt/1 - квадратен корен
- :math.pow/2 - степен

Изход

```
Square root: 5.0
Power: 8.0
```

3. Модул :timer – работа с време

Позволява забавяне и отмерване на време.

Пример

```
defmodule TimerExample do
  def run do
    IO.puts("Start")
  end
end
```

```
:timer.sleep(2000)
IO.puts("After 2 seconds")
end
end
```

```
TimerExample.run()
```

sleep приема милисекунди

Изход

```
Start
(изчакване 2 секунди)
After 2 seconds
```

4. Модул :os – операционна система

Дава информация за системата и променливи на средата.

Пример

```
defmodule OsExample do
  def run do
    system = :os.type()
    time = :os.system_time()
    IO.inspect(system)
    IO.puts("System time: #{time}")
  end
end
OsExample.run()
```

:os.type() връща {операционна система, вариант}

Изход

```
{:win32, :nt}
System time: 17730816665060000
```

Функцията :os.system_time() в Elixir връща системното време като цяло число, измерено в наносекунди (nanoseconds).

Какво означава резултатът, който върна при мен за System time?

```
System time: 17730816665060000
```

Това е брой наносекунди отмерени от началото на системния часовник (Unix време), т.е. времето изминало от 1 януари 1970 г.

Мерни единици:

1 секунда = 1 000 000 000 наносекунди

1 милисекунда = 1 000 000 наносекунди

Как да го преобразуваме в секунди?

Пример

```
time = :os.system_time()
seconds = time / 1_000_000_000
IO.puts(seconds)
```

Функцията позволява директно да зададете единица:

```
:os.system_time(:second)
```

или

```
:os.system_time(:millisecond)
```

Пример

```
time = :os.system_time(:second)
IO.puts("Seconds: #{time}")
```

Този системен таймер е много точен - 17 730 816 665 060 000 и означава:

- време от Unix epoch
- измерено в наносекунди

В Elixir по-често се използва: System.system_time(), защото е по-модерният интерфейс

Да се преобразува числото в низ и да се запише

Алгоритъм I

- 1) Числото се превръща в стринг.
- 2) Разделя се на групи по 3 цифри (хиляди).
- 3) Всяка група се превръща в думи.
- 4) Добавя се съответната единица:

- хиляди
- милиона
- милиарда
- трилиона
- квадрилона

17 | 730 | 816 | 665 | 060 | 000

квадрилона | трилиона | милиарда | милиона | хиляди

Алгоритъм II

- 1) Делим числото на стотици, десетици и единици.
- 2) Ако числото е над 1000, разделяме го на:
 - хиляди
 - остатък
- 3) Извикваме функцията рекурсивно.

Примерна реализация

```
defmodule NumberToWords do
  @ones ["", "едно", "две", "три", "четири", "пет", "шест", "седем", "осем", "девет"]
  @teens ["десет", "единадесет", "дванадесет", "тринадесет",
    "четиринадесет", "петнадесет", "шестнадесет",
    "седемнадесет", "осемнадесет", "деветнадесет"]
  @tens ["", "", "двадесет", "тридесет", "четиридесет",
    "петдесет", "шестдесет", "седемдесет",
    "осемдесет", "деветдесет"]
  @hundreds ["", "сто", "двеста", "триста", "четиристотин",
    "петстотин", "шестстотин", "седемстотин",
    "осемстотин", "деветстотин"]

  def convert(n) when n < 1000 do
    h = div(n, 100)
    rest = rem(n, 100)
    cond do
      rest < 10 ->
        "#{Enum.at(@hundreds, h)} #{Enum.at(@ones, rest)}"
      rest < 20 ->
        "#{Enum.at(@hundreds, h)} #{Enum.at(@teens, rest-10)}"
      true ->
        t = div(rest, 10)
        o = rem(rest, 10)
        "#{Enum.at(@hundreds, h)} #{Enum.at(@tens, t)} #{Enum.at(@ones, o)}"
    end
  end

  def convert(n) when n < 1_000_000 do
    thousands = div(n, 1000)
```

```

rest = rem(n, 1000)
cond do
  thousands == 0 ->
    convert(rest)
  rest == 0 ->
    "#{convert(thousands)} хиляди"
  true ->
    "#{convert(thousands)} хиляди #{convert(rest)}"
end
end
end
IO.puts NumberToWords.convert(2345)

```

5. Модул :rand – случайни числа

Генерира псевдослучайни стойности.

Пример

```

defmodule RandExample do
  def run do
    num = :rand.uniform(10)

    IO.puts("Random number: #{num}")
  end
end

RandExample.run()
uniform(10) - число между 1 и 10

```

Изход (пример)

```
Random number: 7
```

ЗАДАЧА. Да се създаде програма, която генерира случайно число между 1 и 100. Потребителят трябва да го отгатне за 30 секунди, като таймерът започва при генерирането на числото и брой от 1 до 30. Ако времето изтече, програмата съобщава, че числото не е отгатнато и предлага нова игра.

```

defmodule GuessNumberTimer do
  def start do
    number = :rand.uniform(100)
    start_time = System.monotonic_time(:second)
    IO.puts("Guess the number between 1 and 100. You have 30 seconds.")
    guess(number, start_time)
  end

  def guess(number, start_time) do
    current_time = System.monotonic_time(:second)
    elapsed = current_time - start_time

    if elapsed >= 30 do
      IO.puts("Time is up! You didn't guess the number.")
      new_game()
    else
      input = IO.gets("Enter number: ") |> String.trim()
      {guess, _} = Integer.parse(input)

      cond do
        guess < number ->
          IO.puts("Too small!")
          guess(number, start_time)
        guess > number ->

```

```

    IO.puts("Too big!")
    guess(number, start_time)
  true ->
    IO.puts("Congratulations! You guessed the number!")
    new_game()
  end
end
end

def new_game do
  answer = IO.gets("Play again? (y/n): ") |> String.trim()
  if answer == "y" do
    start()
  else
    IO.puts("Goodbye!")
  end
end
end

GuessNumberTimer.start()

```

Запис	Пояснение
<code>defmodule GuessNumberTimer do</code>	Дефинира нов модул с име <code>GuessNumberTimer</code> .
<code>def start do</code>	Дефинира публична функция <code>start</code> , която служи за започване на играта.
<code>number = :rand.uniform(100)</code>	Генерира случайно число между 1 и 100.
<code>start_time = System.monotonic_time(:second)</code>	Записва момента на започване на играта в секунди.
<code>IO.puts("Guess the number between 1 and 100. You have 30 seconds.")</code>	Извежда инструкция за играча.
<code>guess(number, start_time)</code>	Извиква функцията за отгатване и подава числото и началното време.
<code>end</code>	Край на функцията <code>start</code> .
<code>def guess(number, start_time) do</code>	Дефинира функцията, която обработва опитите за отгатване.
<code>current_time = System.monotonic_time(:second)</code>	Взема текущото системно време.
<code>elapsed = current_time - start_time</code>	Изчислява колко секунди са минали от началото на играта.
<code>if elapsed >= 30 do</code>	Проверява дали са изминали 30 секунди.
<code>IO.puts("Time is up! You didn't guess the number.")</code>	Извежда съобщение, че времето е изтекло.
<code>new_game()</code>	Предлага започване на нова игра.
<code>else</code>	Ако времето не е изтекло, програмата продължава.
<code>input = IO.gets("Enter number: ") > String.trim()</code>	Въвеждаме ново число
<code>{guess, _} = Integer.parse(input)</code>	Преобразува въведения текст в цяло число. <code>_</code> игнорира остатъка.
<code>cond do</code>	Започва условна конструкция с няколко проверки.
<code>guess < number -></code>	Проверява дали въведеното число е по-малко.
<code>IO.puts("Too small!")</code>	Извежда подсказка, че числото е по-малко.
<code>guess(number, start_time)</code>	Рекурсивно извиква функцията за нов опит.
<code>guess > number -></code>	Проверява дали въведеното число е по-голямо.
<code>IO.puts("Too big!")</code>	Извежда подсказка, че числото е по-голямо.
<code>guess(number, start_time)</code>	Позволява нов опит.
<code>true -></code>	Ако числото е познато.
<code>IO.puts("Congratulations! You guessed the</code>	Извежда съобщение за успех.

<code>number!")</code>	
<code>new_game()</code>	Предлага нова игра.
<code>end</code>	Край на <code>cond</code> .
<code>end</code>	Край на <code>if</code> .
<code>end</code>	Край на функцията <code>guess</code> .
<code>def new_game do</code>	Дефинира функция за започване на нова игра.
<code>answer = IO.gets("Play again? (y/n): ")> String.trim()</code>	Подава съобщение за стартиране на нова игра
<code>if answer == "y" do</code>	Проверява дали отговорът е <code>y</code> .
<code>start()</code>	Стартира нова игра.
<code>else</code>	Ако отговорът не е <code>y</code> .
<code>IO.puts("Goodbye!")</code>	Извежда съобщение за край.
<code>end</code>	Край на условието.
<code>end</code>	Край на функцията <code>new_game</code> .
<code>end</code>	Край на модула.
<code>GuessNumberTimer.start()</code>	Стартира програмата.

6. Модул `:lists` – работа със списъци

Много полезен модул от Erlang.

Пример

```
defmodule ListsExample do
  def run do
    list = [1,2,3,4]

    sum = :lists.sum(list)
    reversed = :lists.reverse(list)

    IO.puts("Sum: #{sum}")
    IO.inspect(reversed)
  end
end
ListsExample.run()
```

- `:lists.sum/1` - сума
- `:lists.reverse/1` - обръща списъка

Изход

```
Sum: 10
[4,3,2,1]
```

7. Модул `:string` – работа с текст

Функции за текстова обработка.

Пример

```
defmodule StringExample do
  def run do
    text = 'hello'
    length = :string.len(text)
    IO.puts("Length: #{length}")
  end
end
StringExample.run()
```

Erlang използва `charlist 'hello'`

Изход

```
Length: 5
```

ЗАДАЧА. Да се въведе текст от клавиатурата и фраза за търсене. Програмата трябва да изведе колко пъти фразата се среща в текста.

Примерно решение

```
defmodule PhraseCounter do
  def run do
    text = IO.gets("Enter text: ") |> String.trim()
    phrase = IO.gets("Enter phrase to search: ") |> String.trim()
    count = count_phrase(text, phrase)
    IO.puts("The phrase appears #{count} times.")
  end

  def count_phrase(text, phrase) do
    String.split(text, phrase) |> length() |> Kernel.-(1)
  end
end

PhraseCounter.run()
```

Редът

```
String.split(text, phrase) |> length() |> Kernel.-(1)
```

изчислява колко пъти дадена фраза се среща в текста.

length() - Брои колко елемента има в получения списък.

Kernel.-(1) - Изважда 1 от броя на елементите. Това дава броя на срещанията на фразата.

Пример

```
text = "elixir is good elixir is fast"
phrase = "elixir"
```

Разделяне

```
String.split(text, phrase)
```

Резултат

```
["", " is good ", " is fast"]
```

Брой части

```
length(...)
```

Резултат

```
3
```

Изваждане $3 - 1 = 2$. Фразата "elixir" се среща 2 пъти. т.е. ако една дума се среща N пъти, текстът се разделя на $N + 1$ части, затова изваждаме 1.

Как работи String.split/2?

Функцията String.split(text, phrase) разделя текста на части, използвайки търсената фраза като разделител. Всеки път, когато срещне phrase, String.split/2 спира сегашната част и започва нова. Фразата сама по себе си се премахва от резултата.

```
Текстът: "elixir is good elixir is fast"
Фразата: "elixir"
```

Започваме от началото:

- Текстът започва с "elixir". Това води до празна част преди първото срещане на "elixir". Затова първият елемент в списъка е "".
- След първото "elixir", следва " is good " до следващото "elixir". Това става втората част на списъка: " is good ".
- След второто "elixir", остава " is fast". Това е третата част: " is fast".

Резултат на `String.split(text, phrase)`

```
["", " is good ", " is fast"]
```

Брой части

```
length(["", " is good ", " is fast"]) = 3
```

Тъй като ние ТЪРСИМ колко пъти се среща самата фраза, използваме `length(...)` - 1, като $3 - 1 = 2$. И това е правилният брой на срещанията на "elixir" в текста.

Защо първият елемент е празен низ ""?

Защото текстът започва със самата фраза. Ако текстът започва с търсената фраза, няма нищо пред нея ще има празен низ.

Пример

```
String.split("elixir is fun", "elixir")  
# => ["", " is fun"]
```

Ако фразата беше в средата, нямаше да има празен низ в началото:

```
String.split("I love elixir", "elixir")  
# => ["I love ", ""]
```

Обобщение

- `String.split(text, phrase)` разделя текста по фразата, премахвайки я.
- Празните низове се появяват, ако текстът започва или завършва с фразата.
- За да получим броя на срещанията на фразата, правим `length(...)` - 1.

8. Модул :file – работа с файлове

Позволява четене и запис.

Пример

```
defmodule FileExample do  
  def run do  
    :file.write_file("test.txt", "Hello")  
    {:ok, content} = :file.read_file("test.txt")  
  
    IO.inspect(content)  
  end  
end  
FileExample.run()
```

резултатът е binary

Изход

```
"Hello"
```

Текстови файлове (.txt)

Форматът е прост: редове текст, разделени с `\n`.

Пример (Elixir):

```
# Записване на текст в файл  
File.write!("example.txt", "Hello\nWorld")  
  
# Четене на текст от файл  
content = File.read!("example.txt")  
IO.puts(content)
```

Може да използвате и `File.stream!` за работа с големи файлове ред по ред.

Word файлове (.doc, .docx)

Структуриран текстов формат на Microsoft Word.

Не са директно четими като .txt, защото съдържат форматиране и метаданни.

За четене/запис се използват библиотеки:

- Python: python-docx
- Node.js: docx
- Elixir: няма директна стандартна библиотека, обикновено се извличат чрез конвертор към .txt или .html.

Пример (Python):

```
from docx import Document
doc = Document("example.docx")
for para in doc.paragraphs:
    print(para.text)
```

Excel файлове (.xls, .xlsx)

Структурирани таблици.

Не са просто текст — имат клетки, листове, форматиране.

За работа се използват специални библиотеки:

- Python пример (чете Excel):

```
import pandas as pd

df = pd.read_excel("example.xlsx")
print(df.head())
```

- Elixir: може да се използва elixlsx или да се конвертира в CSV за по-лесна обработка.

CSV файлове (.csv)

Кома-сепарирани стойности (или с друг разделител).

Много удобни за импортиране/експортиране на таблици.

Четивни от текстови редактори и Excel.

Пример (Elixir):

```
{:ok, content} = File.read("example.csv")
rows = String.split(content, "\n")
Enum.each(rows, fn row -> IO.inspect(String.split(row, ",")) end)
```

Python:

```
import csv

with open("example.csv", newline="") as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

Бинарни файлове (.dat, .bin и др.)

- Съдържат данни в нечитаем за човек формат.
- Могат да съдържат числа, изображения, аудио и т.н.
- Четенето и писането изисква да се работи в binary mode.

Пример (Elixir):

```
# Запис на бинарни данни
File.write!("example.dat", <<1, 2, 3, 255>>)

# Четене на бинарни данни
{:ok, binary} = File.read("example.dat")
IO.inspect(binary) # => <<1, 2, 3, 255>>
```

Python:

```
# Запис на бинарни данни
with open("example.dat", "wb") as f:
    f.write(b'\x01\x02\x03\xff')

# Четене на бинарни данни
```

```
with open("example.dat", "rb") as f:
    data = f.read()
    print(data) # b'\x01\x02\x03\xff'
```

Универсален пример на Elixir, който показва как се работи с различни типове файлове: .txt, .csv, .dat (бинарен). За .doc/.xlsx няма стандартни Elixir модули, но ще включим пример как да се обработят чрез конвертиране към текст или CSV.

```
defmodule FileExamples do
  @moduledoc """
  Пример за основни операции с файлове в Elixir
  - .txt
  - .csv
  - .dat (бинарен)
  - Принципно за .doc/.xlsx: четене чрез конвертиране
  """

  ## Работа с текстов файл (.txt)
  def text_file_example do
    filename = "example.txt"

    # Записваме текст
    File.write!(filename, "Hello Elixir!\nThis is a text file.\n")

    # Четене на целия текст
    content = File.read!(filename)
    IO.puts("Съдържание на txt файла:\n#{content}")

    # Четене ред по ред
    IO.puts("Ред по ред:")
    File.stream!(filename)
    |> Enum.each(&IO.puts("-> #{String.trim(&1)}"))
  end

  ## Работа с CSV файл (.csv)
  def csv_file_example do
    filename = "example.csv"

    # Създаваме CSV текст
    csv_text = "Name,Age,City\nAlice,30,Sofia\nBob,25,Plovdiv\n"
    File.write!(filename, csv_text)

    # Четене на CSV ред по ред
    IO.puts("\nСъдържание на CSV файла:")
    File.stream!(filename)
    |> Enum.each(fn line ->

      # Премахваме нов ред и разделяме по запетая
      fields = String.split(String.trim(line), ",")
      IO.inspect(fields)
    end)
  end

  ## Работа с бинарен файл (.dat)
  def binary_file_example do
    filename = "example.dat"

    # Създаваме бинарни данни
    binary_data = <<1, 2, 3, 255, 100>>
    File.write!(filename, binary_data)
  end
end
```

```

# Четене на бинарния файл
{:ok, read_data} = File.read(filename)
IO.puts("\nБинарно съдържание на dat файла: #{inspect(read_data)}")
end

## Word и Excel файлове
def doc_xlsx_note do
  IO.puts("""
За .doc/.docx файлове:
    - Няма стандартен Elixir модул.
    - Можете да конвертирате документа в .txt или .csv и след това да го четете.

За .xls/.xlsx файлове:
    - Можете да използвате библиотеката elixlsx (https://hex.pm/packages/elixlsx) за създаване/четене.
    - Или конвертирайте Excel в CSV и го обработете както горе.
  """)
end

## Основна функция, която извиква всички примери
def run_all do
  text_file_example()
  csv_file_example()
  binary_file_example()
  doc_xlsx_note()
end
end

# Изпълнение на примерите
FileExamples.run_all()

```

Какво прави този код?

.txt файл - Записва текст, чете целия файл и ред по ред.
.csv файл - Записва CSV данни, чете ред по ред и разделя колоните.
.dat файл (бинарен) - Записва бинарни данни и ги чете обратно.
.doc и .xlsx

Текстови файлове (.txt)

Таблица 1. Основни функции

Функция	Какво прави	Пример
File.write/2	Записва текст в файл (създава/заменя)	File.write!("file.txt", "Hello")
File.read/1	Чете съдържанието на файл	{:ok, content} = File.read("file.txt")
File.stream!/1	Чете ред по ред (стрийм)	File.stream!("file.txt")
File.exists?/1	Проверява дали файлът съществува	File.exists?("file.txt")
File.rm/1	Изтрива файл	File.rm("file.txt")
File.append/2	Добавя текст в края на файл	File.write!("file.txt", "New line", [:append])
File.rename/2	Преименува файл	File.rename("file.txt", "new_file.txt")

CSV файлове (.csv)

CSV е текстов файл, така че всички функции за .txt работят. Обикновено се обработва ред по ред и се разделя на колони.

Пример функции за CSV

```

# Четене ред по ред
File.stream!("example.csv")
|> Enum.each(fn line ->
  fields = String.split(String.trim(line), ",") # разделяне по запетая

```

```

IO.inspect(fields)
end)

# Добавяне на ред
File.write!("example.csv", "Charlie,40,Burgas\n", [:append])

```

Можете да използвате библиотеки като NimbleCSV за по-сложна обработка.

Бинарни файлове (.dat, .bin)

Таблица 2. Основни функции

Функция	Какво прави	Пример
File.write/2 с бинарен режим	Записва бинарни данни	File.write!("file.dat", <<1,2,3,255>>)
File.read/1	Чете бинарни данни	{:ok, binary} = File.read("file.dat")
File.append/2	Добавя бинарни данни	File.write!("file.dat", <<4,5>>, [:append])
File.exists?/1	Проверява съществуване	File.exists?("file.dat")
File.rm/1	Изтриване	File.rm("file.dat")

В Elixir няма специален binary mode за четене — всичко е бинарно по подразбиране, когато се използва File.read/1.

Word файлове (.doc/.docx)

Elixir няма стандартна библиотека за директна работа с Word.

Подход: конвертиране в .txt или .csv и обработка както горе.

Примерна идея (псевдо-Elixir)

```

# 1. Конвертираме example.docx -> example.txt (чрез LibreOffice, Python или друг инструмент)
# 2. Четем текста
content = File.read!("example.txt")
IO.puts(content)

```

Excel файлове (.xls/.xlsx)

Elixir стандартно не чете .xlsx.

Възможности

- Конвертиране в CSV и четене с File.stream!/1.
- Използване на библиотеката elixlsx
- за създаване/четене на Excel файлове.

Пример с CSV (Excel -> CSV):

```

File.stream!("example.csv")
> Enum.each(fn line ->
  IO.inspect(String.split(String.trim(line), ","))
end)

```

Универсални файлови операции

Таблица 3.

Функция	За кои файлове	Описание
File.exists?/1	всички	проверка за съществуване
File.rm/1	всички	изтриване
File.rename/2	всички	преименуване
File.cp/2	всички	копиране
File.ls/1	директория	изброява файлове в папка
File.mkdir/1	директория	създаване на папка

- Текстови и CSV файлове се обработват еднакво.
- Бинарни файлове се третират със същите функции, но не се използват String.split/1.

- За .doc/.docx и .xlsx обикновено се прави конверсия в текст/CSV и после се чете.

9. Модул :erlang – основни функции на виртуалната машина

Това е най-ниското ниво на BEAM.

Пример

```
defmodule ErlangExample do
  def run do
    pid = self()
    IO.inspect(pid)
  end
end
```

```
ErlangExample.run()
```

self() връща PID на текущия процес

Изход

```
#PID<0.96.0>
```

Други често използвани модули при работа с процеси, concurrency, binary данни и системна информация.

Какво е PID?

PID (Process Identifier) е уникален идентификатор на процес в Elixir/Erlang, чрез който процесите могат да се намират и да си изпращат съобщения.

Пример

```
pid = spawn(fn -> IO.puts("Hello") end)
IO.inspect(pid)
```

spawn създава нов процес. Променливата pid съдържа идентификатора на този процес.

Примерен изход

```
#PID<0.105.0>
```

```
Hello
```

#PID<0.105.0> е адресът на процеса в BEAM виртуалната машина и се използва при send(pid, message) за изпращане на съобщения между процеси.

10. Модул :erlang за създаване на процеси

Позволява създаване на нови паралелни процеси.

Пример

```
defmodule ProcessExample do
  def say_hello do
    IO.puts("Hello from another process")
  end

  def run do
    :erlang.spawn(fn -> say_hello() end)

    IO.puts("Main process finished")
  end
end
```

```
ProcessExample.run()
```

spawn създава нов процес. Процесите в Elixir са леки и независими

Изход

```
Main process finished
```

```
Hello from another process
```

В контекста на Elixir/Erlang, един процес е самостоятелен, изолиран изпълним поток на код, който има собствено състояние и памет и комуникира с други процеси чрез съобщения.

Основни характеристики

- Леки и бързи – процесите в Elixir са много по-леки от системните ОС процеси, затова може да има милиони наведнъж.
- Изолирани – грешката в един процес не влияе върху другите.
- Комуникират чрез съобщения – процесите не споделят пряко памет, а обменят информация чрез send и receive.
- Идентифицират се с PID – всеки процес има уникален Process Identifier (PID).

Пример

```
pid = spawn(fn ->
  IO.puts("Hello from a process")
end)
```

spawn създава нов процес. Кодът вътре се изпълнява паралелно с основния поток. pid е идентификаторът на процеса, който може да се използва за изпращане на съобщения.

Процесът е основната единица на паралелност в Elixir и е ключова за concurrency, fault-tolerance и OTP архитектурата. Процесите в Elixir се използват за паралелно изпълнение на задачи и безопасна комуникация между части на програмата чрез изпращане на съобщения.

Основно предназначение

- **Паралелно изпълнение (Concurrency).** Позволяват много задачи да се изпълняват едновременно.
- **Изолираност на грешките.** Всеки процес има собствена памет, така че грешка в един процес не спира останалите.
- **Комуникация чрез съобщения.** Процесите обменят информация чрез send и receive.
- **Скалируемост.** Elixir може да управлява милиони леки процеси.

Пример

```
pid = spawn(fn ->
  receive do
    msg -> IO.puts("Received: #{msg}")
  end
end)
send(pid, "Hello")
```

Създава се нов процес, който чака съобщение и го отпечатва, когато бъде получено.

11. Модул :binary – работа с binary данни

Използва се за ниско ниво обработка на binary данни.

Пример

```
defmodule BinaryExample do
  def run do
    data = "hello"
    first = :binary.first(data)
    last = :binary.last(data)
    IO.puts("First char code: #{first}")
    IO.puts("Last char code: #{last}")
  end
end

BinaryExample.run()
```

first връща ASCII код

Изход

```
First char code: 104
Last char code: 111
```

12. Модул :maps – работа с map структури

Erlang има собствен модул за работа с map.

Пример

```
defmodule MapsExample do
  def run do
    map = %{name: "Ivan", age: 25}
    keys = :maps.keys(map)
    values = :maps.values(map)
    IO.inspect(keys)
    IO.inspect(values)
  end
end
```

MapsExample.run()

- **keys/1** - връща ключовете
- **values/1** - връща стойностите

Изход

```
[:name, :age]
["Ivan", 25]
```

13. Модул :ets – in-memory база данни

Erlang Term Storage (ETS) е много бърза in-memory база и се използва за кеширане, споделяне на състояние между процеси и ETS (Erlang Term Storage) е таблица в паметта, която позволява бързо съхранение и достъп на данни.

Основни характеристики

- In-memory – всички данни се пазят в RAM, което прави достъпа много бърз.
- Поддържа различни типове таблици, като:
 - :set – уникални ключове
 - :bag – ключовете могат да имат няколко стойности
 - :duplicate_bag – позволява повторения на еднакви ключ-стойност двойки
- Достъпно от няколко процеса – таблиците могат да бъдат :public, :protected или :private.
- Не е персистентно – данните се губят при спиране на VM.

Пример

```
defmodule EtsExample do
  def run do
    # Създаване на ETS таблица
    table = :ets.new(:users, [:set, :public])

    # Вмъкване на данни
    :ets.insert(table, {:user1, "Ivan"})
    :ets.insert(table, {:user2, "Maria"})

    # Вземане на данни
    result = :ets.lookup(table, :user1)

    IO.inspect(result)
  end
end
```

EtsExample.run()

Изход

```
[user1: "Ivan"]
```

или

```
[{:user1, "Ivan"}]
```

14. Модул :crypto – криптография

Използва се за hash функции и криптиране.

SHA-256 е криптографска хеш функция, която преобразува всякакви входни данни в фиксиран 256-битов хеш.

Принципът на работа е

- Входът се разделя на блокове и се допълва до кратен 512 бита.
- Всеки блок се обработва последователно чрез сложни битови операции, логически функции и константи.
- Резултатът от всяка итерация се комбинира с предишния, за да се получи окончателният хеш.
- Малка промяна във входа води до драстично различен хеш (property avalanche).
- Хешът е еднопосочен, т.е. не може лесно да се възстанови оригиналният вход.

Пример

```
defmodule CryptoExample do
  def run do
    hash = :crypto.hash(:sha256, "hello")
    IO.inspect(hash)
  end
end
```

```
CryptoExample.run()
```

:sha256 е hash алгоритъм

Изход

```
<<44, 242, 77, 186, ... >>
```

(binary hash)

15. Модул :base64 – кодиране

Позволява Base64 encoding и decoding.

Пример

```
defmodule Base64Example do
  def run do
    encoded = :base64.encode("hello")
    IO.inspect(encoded)
  end
end
```

```
Base64Example.run()
```

Изход

```
"aGVsbG8="
```

Как работи?

16. Модул :calendar – работа с дати

Позволява работа с дата и време.

Пример

```
defmodule CalendarExample do
  def run do
    date = :calendar.local_time()
    IO.inspect(date)
  end
end
```

```
CalendarExample.run()
```

Изход

```
{{2026,3,6},{14,30,10}}
```

16. Модул :queue – опашка (FIFO)

Полезен за структури от данни.

Пример

```
defmodule QueueExample do
  def run do
    q = :queue.new()
    q = :queue.in(1, q)
    q = :queue.in(2, q)
    {{:value, first}, q} = :queue.out(q)
    IO.puts("First element: #{first}")
  end
end

QueueExample.run()
```

Важна особеност!

В Erlang/Elixir има вградени структури **за стек**, макар че няма отделен модул Stack, както при опашката (:queue). Основните възможности са:

- **Списъци (List) като стек**

В Elixir/Erlang списъкът [head | tail] естествено работи като стек, защото добавянето и премахването на елемент от началото е O(1).

Пример

```
stack = []
# push
stack = [1 | stack]
stack = [2 | stack]
# stack = [2, 1]

# pop
[top | stack] = stack
top # => 2
stack # => [1]
```

[head | tail] - премахва/взема елемент от върха. Списъците в Elixir се използват като стек по подразбиране.

- **Модул :queue**

:queue поддържа FIFO и LIFO:

- :queue.in/2 и :queue.out/1 – за опашка (FIFO)
- :queue.in_r/2 и :queue.out_r/1 – за стек (LIFO от дясната страна)

Пример (стек с :queue)

```
q = :queue.new()
q = :queue.in_r(1, q) # push
q = :queue.in_r(2, q) # push
{{:value, top}, q} = :queue.out_r(q) # pop
top # => 2
```

:queue е по-гъвкав, поддържа и опашки, и стекове.

- **ETS (Erlang Term Storage) / Map / List**

Може да се имплементира стек чрез ETS или Map, но това е по-рядко. Най-често се използват списъци или :queue.

Опашката (:queue) в Erlang е двустранна структура, която позволява да се използва и като FIFO, и като LIFO, в зависимост от функциите, които използваме.

FIFO (First In, First Out) – стандартно поведение на опашка:

- :queue.in/2 - добавя елемент в края
- :queue.out/1 - премахва елемент от началото

```
q = :queue.new()
q = :queue.in(1, q)
q = :queue.in(2, q)
{{:value, first}, q} = :queue.out(q)
first #=> 1 (първо добавеният)
```

LIFO (Last In, First Out) – поведение на стек:

- :queue.in_r/2 - добавя елемент в началото
- :queue.out_r/1 - премахва елемент от началото

```
q = :queue.new()
q = :queue.in_r(1, q)
q = :queue.in_r(2, q)
{{:value, top}, q} = :queue.out_r(q)
top #=> 2 (последно добавеният)
```

:queue е двустранна опашка (double-ended queue / deque). Затова:

- Използваме стандартни in/out - FIFO
- Използваме in_r/out_r - LIFO

т.е, опашката може да работи и като стек, без да се създава отделна структура.

Категория	Erlang модул	Предназначение
Процеси	:erlang	процеси и VM функции
Binary	:binary	binary обработка
Map	:maps	работа с map
In-memory DB	:ets	бърза база в памет
Криптография	:crypto	hash и криптиране
Encoding	:base64	Base64 кодиране
Дата и време	:calendar	работа с дати
Структури	:queue	опашки

Много важно! Една от големите сили на Elixir е, че може директно да използва всички Erlang библиотеки, което означава:

- хиляди готови модули
- OTP concurrency модели
- висока надеждност

5. Обхват (Scope) в Elixir

Обхватът определя къде са видими променливите и функциите.

- **Лексикален обхват** – валиден в рамките на блок.

В Elixir променливите са видими само в рамките на блока, в който са дефинирани. Това се нарича лексикален обхват.

Пример

```
defmodule ScopeExample do
  def demo do
    x = 10
    IO.puts("x вътре: #{x}")
    if true do
      y = 20
      IO.puts("x вътре в if: #{x}") # x се вижда
    end
  end
end
```

```
IO.puts("у вътре в if: #{y}")
end
# IO.puts(y) # грешка, у не е видима извън if
end
end
ScopeExample.demo()
```

Изход:

```
x вътре: 10
x вътре в if: 10
у вътре в if: 20
```

Разлика спрямо Erlang:

В Erlang също има лексикален обхват, но променливите не могат да бъдат **“преназначавани”**, докато в Elixir можем да „затъмняваме“ (shadow) съществуващи променливи.

Преназначаване (reassignment)

Преназначаване означава една и съща променлива да получи нова стойност след като вече има стара.

Пример

```
x = 5
x = 10
```

Тук променливата x първо има стойност 5, а после 10 - това е преназначаване.

Как е в Erlang?

В Erlang променливите са immutable и single assignment. Това означава: променлива може да получи стойност само веднъж, след това не може да бъде променена.

Пример

```
X = 5.
X = 10.
```

Това ще даде грешка, защото X вече има стойност 5.

В Erlang вторият ред всъщност означава:

- "Провери дали X е 10"
- а не "направи X = 10".

Как е в Elixir?

В Elixir променливите също са immutable, но езикът позволява shadowing. Shadowing означава - създава се нова променлива със същото име, която скрива старата.

Пример

```
x = 5
x = 10
IO.puts(x)
```

Резултат:

```
10
```

Тук не се променя старото x. Създава се ново x, което затъмнява (shadow) старото.

Какво значи това на практика?

Elixir

```
x = 5
x = x + 1
IO.puts(x)
```

Резултат:

6

Старото `x = 5` остава непроменена, а новото `x` има стойност 6.

В блокове (още по-ясно)

```
x = 10
if true do
  x = 20
  IO.puts("в блока: #{x}")
end
IO.puts("извън блока: #{x}")
```

Резултат:

- в блока: 20
 - извън блока: 10
- т.е.
- вътре в блока има ново `x`
 - старото `x` извън блока остава 10

- **Засенчване (shadowing)** – променлива може да бъде предефинирана.

Можете да дефинирате променлива със същото име в по-вътрешен блок, което замества старата променлива в този блок.

Пример

```
x = 5
IO.puts("x = #{x}")
x = x + 2 # shadowing, x се предефинира
IO.puts("ново x = #{x}")
```

Изход:

```
x = 5
ново x = 7
```

Разлика спрямо Erlang:

В Erlang няма обичайно „преназначаване“ на променливи. Всяка променлива е еднократна, опит за „преназначаване“ води до грешка.

- **Обхват на функции и модули**

Функциите имат локален обхват на аргументите и локалните променливи. Модулните дефиниции определят глобален обхват в рамките на модула.

Пример

```
defmodule MyModule do
  def add(a, b) do
    sum = a + b
    sum
  end

  def double(x) do
    add(x, x)
  end
end

IO.puts(MyModule.double(3)) # => 6
# IO.puts(sum) # грешка, sum не е видима извън add/2
```

Разлика спрямо Erlang:

Същото, но Elixir позволява по-удобен синтаксис и `private` функции (`defp`), което ограничава обхвата на функцията.

- **case, try и comprehensions**

- **case**: създава локален блок, където променливите са видими само вътре.
- **try**: създава собствен обхват за rescue/after/else.
- **comprehensions**: променливите, дефинирани вътре, са видими само в блока на comprehension.

Пример с case

Напишете код, който използва case за проверка на стойността на променлива *x* и създава локална променлива *y* само вътре в клаузата. Опитайте след това да отпечтатате *y* извън case блока и обяснете защо се получава грешка.

```
x = 10
case x do
  10 ->
    y = x * 2
    IO.puts("y = #{y}")
  _ ->
    IO.puts("не е 10")
end
# IO.puts(y) # грешка, y е само в блока case
```

- Променливата *x* е дефинирана **извън блока** и има стойност **10**.
- *x* е видима навсякъде в текущия лексикален scope.
- Вътре в **case** блока **имаме клауза**:

```
10 ->
  y = x * 2
  IO.puts("y = #{y}")
```

- Тук се дефинира променлива *y*, която е локална за тази клауза на case.
- Т.е. *y* съществува само, докато се изпълнява този блок.
- След като case приключи, локалните променливи на клаузите изчезват.

```
# IO.puts(y) # грешка
```

Тук се опитваме да използваме *y*, но тя никога не е била дефинирана извън блока. Затова Elixir дава грешка: variable *y* is undefined.

Лексикален scope на demo():

```
x = 10
case x do
  10 ->
    y = x * 2    y съществува само тук
    IO.puts("y = #{y}")
  _ ->
    y вече не съществува тук
    IO.puts("не е 10")
end
```

Това е стандартното правило за лексикален обхват (lexical scope) в Elixir.

- Case блоковете имат собствен локален обхват.
- Променливи дефинирани вътре са видими само вътре.

Ако искате *y* да се използва извън case, трябва да го дефинирате преди блока:

```
y = nil    # дефинираме извън case

case x do
  10 -> y = x * 2
  _ -> y = 0
end

IO.puts(y)    # сега работи
```

Пример с try

Напишете код, който използва try/rescue за обработка на грешки и променя стойността на променлива **x** вътре в блока чрез **shadowing**. След това отпечатайте **x** и опитайте да използвате локална променлива **y** от **rescue** блока **извън try**, като обясните защо се получава грешка.

```
defmodule TryScopeExample do
  def demo do
    x = 10
    IO.puts("x преди try: #{x}")

    try do
      x = x + 5          # shadowing вътре в try
      IO.puts("x вътре в try: #{x}")

      # симулираме грешка
      raise "Нещо се обърка"
    rescue
      e ->
        IO.puts("В rescue блока, грешка: #{e.message}")
        y = 100
        IO.puts("y в rescue: #{y}")
    end

    IO.puts("x след try: #{x}")
    # IO.puts(y)          # грешка, y не е видима извън rescue
  end
end

TryScopeExample.demo()
```

- **x = 10** се дефинира извън try и е видима в целия метод.
- В блока try се прави **x = x + 5** – това е shadowing, променливата **x** вътре в try блок замества външната само временно в рамките на блока.
- **raise "Нещо се обърка"** симулира грешка, която се улавя от rescue.
- В rescue се дефинира променлива **y = 100**, която е локална само за rescue блока.
- След try стойността на външната **x** остава 10, а опитът да се използва **y** извън блока води до грешка, защото **y** не съществува извън rescue.

Изход

```
x преди try: 10
x вътре в try: 15
В rescue блока, грешка: Нещо се обърка
y в rescue: 100
x след try: 10
```

- Обхватът във try и rescue е локален.
- Shadowing променя променливата само вътре в блока.
- Променливи дефинирани в rescue (**y**) не се виждат извън блока.

Извод: Променливите, дефинирани в try и rescue, имат локален обхват, а shadowing променя само локалната версия на променливата вътре в блока.

Пример с comprehension

В Elixir comprehension е конструкция за генериране и трансформация на колекции, като списъци, MapSet или други Enumerable структури, чрез кратък и ясен синтаксис.

```
squares = for n <- 1..3, do: n * n
IO.inspect(squares) # => [1, 4, 9]
```

- **Разлики спрямо Erlang**
 - В Elixir и Erlang съществува лексикален обхват, т.е. променливите са видими само в блока, в който са дефинирани.
 - Основната разлика е, че в Erlang променливите са с еднократно присвояване (single assignment) и не могат да получат нова стойност след като са дефинирани.

- В Elixir също има неизменяемост (immutability), но езикът позволява shadowing, при което се създава нова променлива със същото име, която временно скрива старата.
- В Erlang опит за повторно присвояване се интерпретира като pattern matching, а не като промяна на стойността, и води до грешка ако стойностите не съвпадат.
- В Elixir блокове като if, case, try и comprehension създават локален обхват, в който дефинираните променливи не са достъпни извън блока.
- Elixir предоставя и по-удобни механизми за контрол на обхвата чрез частни функции (defp) и модулна структура, което улеснява организацията на кода в сравнение с Erlang.

Обобщение

Elixir използва модули, функции и лексикален обхват за изграждане на чист, структуриран и поддържан код. Основните механизми като pattern matching, директиви и ясни правила за обхват улесняват разработката на надеждни приложения.